

Contents

Análise Profunda do Projeto BlazorAppFood (Foodie)	1
Resumo executivo	1
1. Problemas CRÍTICOS de segurança e bugs	1
2. Problemas estruturais	3
3. Problemas funcionais e features incompletas	4
4. Resposta às vossas perguntas	6
5. Plano de trabalho proposto (10-31 maio)	7
6. Recomendações finais	7

Análise Profunda do Projeto BlazorAppFood (Foodie)

Data: 10 de maio de 2026 **Equipa:** 3 pessoas **Deadline:** Final de maio de 2026

Resumo executivo

Li **todos** os ~50 ficheiros relevantes do projeto. O projeto está funcional na superfície mas tem problemas estruturais sérios que precisam de atenção urgente antes da entrega.

Existem **três categorias** de problemas, por ordem de prioridade:

1. **Críticos (segurança e bugs que partem o sistema):** SQL injection generalizada, passwords em texto plano, e um bug de chave de sessão que provavelmente está a causar comportamentos estranhos no login.
 2. **Estruturais (qualidade e manutenção):** services que são na verdade repositórios, models duplicados, ficheiros mortos espalhados pelo projeto, naming inconsistente.
 3. **Funcionais (features incompletas ou partidas):** chat de grupo nunca foi terminado, performance fraca em listagens, NavMenu logout não funciona.
-

1. Problemas CRÍTICOS de segurança e bugs

1.1. SQL Injection — vulnerável em 8+ pontos

Os seguintes métodos constroem queries SQL com interpolação de string usando inputs do utilizador. Qualquer um destes é explorável:

Ficheiro	Método	Risco
Data/LoginService.cs	LogUser	CRÍTICO — login pode ser quebrado com ' OR '1'='1
Data/RegisterService.cs	LoginRegist	CRÍTICO — mesmo bug que LoginService
Data/ChatService.cs	CreateComment	CRÍTICO — comentário do utilizador vai direto para SQL. Aspas no comentário partem o sistema, e atacante pode injetar comandos
Data/ChatService.cs	LoadRecipeComments	Médio (idRecipe é int)
Data/RecipeService.cs	RecipesList	Médio (idUser é int)
Data/RecipeService.cs	RatingRecipe	Médio
Data/RecipeService.cs	AddFavorite	Médio

Ficheiro	Método	Risco
Data/UserService.cs	GetFollowingRecipes	ALTO — emailAddress concatenado direto

Como corrigir: trocar interpolação por parâmetros do Dapper. Vocês já fazem isto noutros métodos:

```
// Errado (atual)
string sQuery = $"Select COUNT(*) from Users Where Email = '{Email}' and Password = '{Password}'";
int validation = conn.ExecuteScalar<int>(sQuery);

// Correto
string sQuery = "SELECT COUNT(*) FROM Users WHERE Email = @Email AND Password = @Password";
int validation = await conn.ExecuteScalarAsync<int>(sQuery, new { Email, Password });
```

1.2. Passwords em texto plano

RegisterService.CreateRegist insere a password tal e qual no campo Password da BD. LoginService.LogUser compara com '{Password}'. Resultado: todas as passwords estão guardadas como texto na BD. Se a BD vazar, todas as contas estão comprometidas.

Solução simples (1 dia de trabalho): 1. Adicionar pacote BCrypt.Net-Next. 2. No CreateRegist: var hash = BCrypt.Net.BCrypt.HashPassword>Password); e guardar hash. 3. No LogUser: buscar o hash do utilizador e validar com BCrypt.Net.BCrypt.Verify>Password, storedHash). 4. Migration: pôr todas as passwords existentes a null e forçar reset.

1.3. BUG GRAVE — Chave de sessão inconsistente

Este é provavelmente a causa de comportamentos estranhos que possam estar a ter no login. Vejam:

- LoginService.LogUser faz await _sessionStorage.SetItemAsync("email", Email); → guarda na chave **email**
- Login.razor (a seguir ao login) faz await sessionStorage.SetItemAsync("emailAddress", user.Email); → guarda também em **emailAddress**
- CustomAuthenticationStateProvider.GetAuthenticationStateAsync lê **emailAddress**
- NavMenu, Feed, RecipePage, Favorites, UserPage, UserPosts, UploadPhoto, AddRecipe — leem **emailAddress**
- **MAS** Feed.razor, GroupPage.razor, GroupDetail.razor leem **email** (a chave antiga!)

Especificamente em Feed.razor linhas 278-289:

```
var email = await sessionStorage.GetItemAsync<string>("email"); // ← chave "email"
var user = await UserService.GetUserByEmail(email);
userGroups = await GroupService.GetUserGroups(user.Id_User); // ← se email==null, dá NF

string emailUser = await sessionStorage.GetItemAsync<string>("emailAddress"); // ← chave "emailAddress"
```

Se um utilizador se registar (que guarda em emailAddress) e depois abrir o feed, os grupos não vão carregar. Se fizer login (que guarda em ambas), funciona por acaso.

Correção: padronizar para uma única chave em **todos** os ficheiros. Sugiro userEmail. Faz um Find & Replace em todo o projeto.

1.4. Bugs adicionais no Login flow

- O LoginService.LogUser valida com if (validation == 1). Se houver dois utilizadores com o mesmo email (o registo não previne isso!), retorna 2 e o login falha.

- O Register não valida se o email já existe antes do INSERT — vai gerar erro de constraint na BD.
- O Register não valida que a “Repetir Password” coincide (o input nem sequer está bound).

2. Problemas estruturais

2.1. “Services” são na verdade Repositórios

A pessoa que vos disse isto está certa. Não há lógica de negócio em lado nenhum nos Data/*Service.cs. Tudo é puro CRUD. A lógica de negócio está espalhada pelos componentes Razor (mau).

Sugestão pragmática (não é refactor grande):

```
Repositories/    ← renomear Data/ para isto, e renomear *Service.cs para *Repository.cs
Services/        ← criar NOVO. Pôr aqui:
    AuthService.cs      (combina Login + Register, faz hash, gere sessão)
    FeedService.cs      (orquestra: vai buscar receitas + tags + favorite count em batch)
Models/
DTOs/            ← criar. Pôr aqui RecipeListDto (sem byte[] de imagem) etc.
```

2.2. PostService está numa namespace diferente

Tudo está em BlazorAppFood.Data exceto PostService que está em BlazorAppFood.Services. Padronizar.

2.3. Models duplicados ou inconsistentes

Problema	Detalhe
Category vs CategoryWithTags	Praticamente iguais — CategoryWithTags tem List<Tag>, Category tem ICollection<Tag>.
User.UserId E User.Id_User	Apagar um. Dois IDs no mesmo model. Só Id_User é usado. Apagar UserId.
Recipe.Ingredients E Recipe.Ingredient	Duas listas iguais. Só Ingredients é usada. Apagar Ingredient.
User.IdHistorico, User.IdPerfil using Azure;	Não usados em lado nenhum. Apagar. Em Category.cs, CategoryWithTags.cs, TagService.cs, ITagService.cs — não usado, sobra de IntelliSense.

2.4. Naming inconsistente

- Id_User (snake) em Users e Recipe vs IdUser em Comments, Posts, Favorites
- IdRecipe em Comments vs Id_Recipe em Recipe
- IdTag (sem underscore) vs Id_Recipe (com underscore)
- Id_Group (snake) vs IdCategory (camel)
- Ficheiros: Add Recipe.razor (com espaço!), Edit Recipe.razor (com espaço!), Delete Recipe.razor (com espaço!) — má prática para C#/Blazor

A BD herdou estas convenções e mudá-las todas é trabalho. **Sugestão pragmática:** documentar e deixar como está, mas pelo menos renomear os ficheiros .razor (sem espaços).

2.5. Duas pastas para componentes

- Components/ — tem RatingComponent.razor (incompleto), RecipePost.razor (placeholder), SearchComponent.razor (placeholder)
- ComponentsFolder/ — tem Rating.razor (este sim, funcional)

Os 3 ficheiros em Components/ são esqueletos vazios sem código real. Só ComponentsFolder/Rating.razor é usado. **Apagar a pasta Components/ inteira e renomear ComponentsFolder/ para Components/.**

2.6. Ficheiros mortos para apagar

- Pages/Counter.razor — template default do Blazor
- Pages/FetchData.razor — template default, depende do WeatherForecastService que está excluído
- Pages/Recipes.razor — placeholder com 12 linhas, não usado
- Pages/AddRecipe.razor — **TODO o ficheiro está comentado**, é a versão antiga; a versão real está em Pages/Add Recipe.razor
- Components/RatingComponent.razor, RecipePost.razor, SearchComponent.razor (mencionados acima)
- Shared/SurveyPrompt.razor
- Data/UserState.cs, WeatherForecast.cs, WeatherForecastService.cs — já estão Compile Remove no csproj
- Models/ManagerResponse.cs — não é usado em lado nenhum

2.7. CSS leakado

Várias páginas têm <style> inline (Feed, RecipePage, ProfilePage, UserPage, UserPosts...). Esses estilos NÃO são scoped ao componente — vazam para o CSS global. Resultado: classes como .container, .sidebar, .btn-red, .modal-content em ProfilePage e UserPage colidem com o resto do app.

Solução: mover para ficheiros .razor.css (CSS isolado) por componente.

3. Problemas funcionais e features incompletas

3.1. Sistema de chat de grupo está PARTIDO

Têm Models/GroupChatHub.cs (um SignalR hub) e Models/GroupChatMessages.cs, e GroupService.GetChatMessages(). Mas:

- O hub **não está registado** no Startup.Configure (falta endpoints.MapHub<GroupChatHub>("/chathub"))
- **Nenhuma página** usa o chat — GroupPage, GroupDetail não têm UI de chat
- O método SendMessage do hub **não persiste** mensagens na BD, só faz broadcast
- Não há método InsertChatMessage em lado nenhum

Conclusão: a feature foi começada mas abandonada. **Decisão a tomar:** ou eliminam a feature (apagar GroupChatHub, GroupChatMessages, GetChatMessages do IGroupService), ou implementam-na a sério (1-2 dias).

3.2. NavMenu — botão “Sair” não funciona

Shared/NavMenu.razor linha 71:

```
<NavLink class="nav-link" href="" ...>
  <i class="bi bi-door-open"></i>Sair
</NavLink>
```

href="" recarrega a página atual. Não há código de logout. **Falta implementar:** limpar session storage, chamar MarkUserAsAuthenticated(null) ou equivalente, redirecionar para /.

3.3. Performance — N+1 queries no Feed

Feed.razor no LoadFeed:

```
foreach (var recipe in listPosts)
{
    recipe.Tags = await RecipeService.GetTagsForRecipe(recipe.Id_Recipe);
    favoriteCount = await RecipeService.GetFavoriteCount(recipe.Id_Recipe);
}
```

- Para 100 receitas, faz **200 queries** à BD.
- Pior: favoriteCount é uma variável **única** (não por receita!), portanto a UI mostra sempre o favoriteCount da última receita do loop em todas as cards. **Bug visível.**

Correção: adicionar FavoriteCount ao Recipe (já existe lá!) e fazer JOIN no SQL. Mesmo para tags.

3.4. Recipe.razor (RecipePage) — mais N+1

```
foreach (var comment in recipe.Comments)
{
    var commentUser = await UserService.GetUserById(comment.IdUser);
    comment.UserName = commentUser?.Username;
    comment.UserPhoto = commentUser?.UserPhoto;
}
```

Mesmo problema. LoadRecipeComments já faz join com Users.UserPhoto — ampliem para incluir Users.Username num só query e o loop desaparece.

3.5. Feed.razor — duplicação de loading

Tem **AMBOS** OnInitializedAsync E OnParametersSetAsync que carregam posts, com lógica diferente. OnParametersSetAsync **vai sempre correr depois** e sobrescreve o resultado. O efeito é que o feed Personalized/Following provavelmente não funciona como esperado quando se entra primeiro em /feed.

3.6. Imagens transmitidas inteiras na listagem

No Feed e em Favorites, cada Recipe.Image (byte[]) é convertido para base64 e enviado em cada card. Para 100 receitas com fotos de 500KB cada, são 50MB de payload por carregamento.

Solução: criar um endpoint GET /api/recipe/{id}/image que sirva a imagem como ficheiro, e nos models de listagem ter só Id e usar . Browser cacheia automaticamente.

3.7. Sem paginação

Feed carrega todas as receitas da BD. Para a apresentação académica não vai ser problema, mas é algo a documentar como “limitação conhecida”.

3.8. Outros bugs menores

- **Login.razor:** o checkbox “Remember Me” não está bound a nada — não faz nada.
- **Register.razor:** campo “Repetir Password” não está bound — não há validação.
- **CategoryPage.razor:** é praticamente todo placeholder (“Cozinha Italiana” hardcoded, listas estáticas).

- **UserPage SavePassword:** passa user.Password diretamente — sem hash. Combinado com o UpdateUser, vai guardar texto plano.
- **GroupService.GetGroupMetadata:** retorna o primeiro admin como “creator” — mas não há campo “creator” na BD, só admins. Se houver mais que um admin, retorna o primeiro arbitrariamente.
- **UserPage Sair:** não implementado.
- **AddRecipe** (Add Recipe.razor): chama RecipeService.GetAllCategoriesAsync e GetAllTagsAsync mas estes métodos estão no RecipeService — deviam estar no TagService. Já têm um TagService.GetCategoriesWithTags que faz o mesmo trabalho de forma mais elegante.
- **PostService:** query ORDER BY DatePublication DESC — confirmem que essa coluna existe na tabela Posts (no diagrama do PDF aparece como DatePublication, parece estar ok).

3.9. SqlConnectionConfiguration

```
public class SqlConnectionConfiguration
{
    public string _value { get; } // ← propriedade pública com underscore-prefix é estranho
}
```

O underscore-prefix é convenção para **campos privados**, não propriedades públicas. Renomear para ConnectionString ou usar IConfiguration direto.

4. Resposta às vossas perguntas

Migração para Entity Framework — vale a pena?

Não, neste prazo. Razões:

1. **Risco/benefício é mau.** EF Core num projeto que mistura stored procedures e SQL inline é trabalhoso. Vão gastar 1-2 semanas só na migração e vão ficar sem tempo para corrigir os bugs reais.
2. **Dapper não é o problema.** O problema é COMO o Dapper está a ser usado (interpolação de strings em vez de parâmetros). Trocar de ORM não resolve isso — só esconde.
3. **Stored procedures que vocês têm na BD ficam órfãos.** Há spRecipe_Insert, spRecipe_GetAll, spIngredients_GetAll, spGetRecipe_GetOne, spRecipe_Delete, spUsers_GetAll. Migrar para EF significa reescrever tudo isso.

Se mesmo assim quiserem EF para mostrar conhecimento: usem só para **uma feature nova** (notificações, por exemplo). Demonstra que sabem usar EF sem partir o que existe.

Os “Services” são realmente repositórios?

Sim, 100%. Não há um único método em Data/*Service.cs que faça lógica de negócio. Tudo é CRUD + um pouco de orquestração de queries.

A “lógica de negócio” está espalhada pelos componentes Razor — por exemplo, Login.razor.LoginUser faz validação + autenticação + sessão + navegação. Isto devia estar num AuthService.

Opinião geral do código

Pontos positivos: - Funcionalidades estão razoavelmente completas (feed, grupos, perfil, edição, favoritos) - Uso correto de EditForm e InputFile do Blazor - AuthorizeView usado corretamente - A maior parte dos services novos (GroupService, partes de UserService) já usa parâmetros do Dapper - Estrutura geral do projeto faz sentido para um projeto académico

Pontos negativos: - Segurança é o calcanhar de Aquiles (SQL injection, password plaintext) - Inconsistências sugerem que múltiplos developers trabalharam sem alinhamento (3 estilos de código

+ o original de 2021) - Muito código morto/comentado por todo o lado - Performance ingénua (N+1, payload pesado) - Features começadas e abandonadas (chat, NavMenu Sair)

5. Plano de trabalho proposto (10-31 maio)

Dividido em 3 sprints de ~1 semana, com tarefas que se podem distribuir entre os 3 elementos.

Sprint 1 (10-17 maio) — Segurança e bugs críticos

Pessoa A — Segurança SQL: - Varrer e corrigir TODAS as SQL injections (lista no §1.1). ~1 dia. - Implementar BCrypt para passwords. Migration de dados se necessário. ~1 dia.

Pessoa B — Login & sessão: - Padronizar chave de sessão para uma só (userEmail). Find & replace em todos os ficheiros. ~0.5 dia. - Implementar logout funcional no NavMenu. ~0.5 dia. - Corrigir validação do Register (email único, repeat password, formato email). ~1 dia.

Pessoa C — Modernização: - Consolidar Startup.cs + Program.cs em um só (estilo .NET 8 minimal). ~0.5 dia. - Atualizar pacotes NuGet (Syncfusion, Dapper, Newtonsoft). ~0.5 dia. - Remover ficheiros mortos (lista §2.6). ~0.5 dia. - Apagar Compile Remove do csproj e os ficheiros que ficaram. ~0.5 dia.

Sprint 2 (17-24 maio) — Estrutura e bugs funcionais

Pessoa A — Refactor estrutural: - Renomear Data/ → Repositories/ e *Service.cs → *Repository.cs. ~1 dia. - Criar Services/AuthService.cs que combina Login+Register+sessão+hash. Mover lógica do Razor para aqui. ~1 dia. - Padronizar PostService no namespace Repositories. ~0.5 dia.

Pessoa B — Performance & bugs: - Corrigir N+1 no Feed (juntar tags + favorite count num só query). ~1 dia. - Corrigir bug do favoriteCount partilhado. ~0.5 dia. - Corrigir N+1 nos comments (LoadRecipeComments com Username). ~0.5 dia. - Corrigir duplicação OnInit/OnParameters no Feed. ~0.5 dia.

Pessoa C — Limpeza de modelos: - Apagar duplicados em models (UserId, Ingredient, CategoryWithTags ou Category). ~0.5 dia. - Mover GetAllCategoriesAsync / GetAllTagsAsync para TagService. ~0.5 dia. - Decidir destino do chat de grupo (apagar ou implementar). ~1 dia. - Mover CSS dos <style> para ficheiros .razor.css isolados. ~1 dia.

Sprint 3 (24-31 maio) — Polish, validação e entrega

- Validação no backend com DataAnnotations ou FluentValidation (registar requer email válido, etc.). ~1 dia.
 - Sistema de notificações (mencionado no PDF “Próximos Passos”). ~2 dias.
 - Polish visual (CategoryPage tem placeholder). ~1 dia.
 - Testes manuais completos (criar checklist do PDF). ~1 dia.
 - Atualizar PDF de apresentação com o novo trabalho. ~0.5 dia.
 - README com instruções de instalação + connection string. ~0.5 dia.
-

6. Recomendações finais

1. **Não migrem para EF.** Foquem em corrigir o que existe.
2. **Renomeiem Data/ para Repositories/** — é honesto sobre o que aquilo é, e não dá trabalho.
3. **Comecem pela segurança.** Se a apresentação correr e algum professor injetar SQL no formulário de login para demonstrar a falha, é mau.

4. **Documentem o que sabem que está limitado** (paginação, etc.) na própria apresentação como “trabalho futuro” — é melhor admitir que esconder.
5. **Rebase no Git frequentemente.** Com 3 pessoas em paralelo nos mesmos ficheiros (especialmente services) os conflitos vão ser dolorosos.

Boa sorte! Estou disponível para ajudar com qualquer correção específica.